

Virtual Memory: Just an Illusion

Hung-Wei Tseng

Let's dig into this code

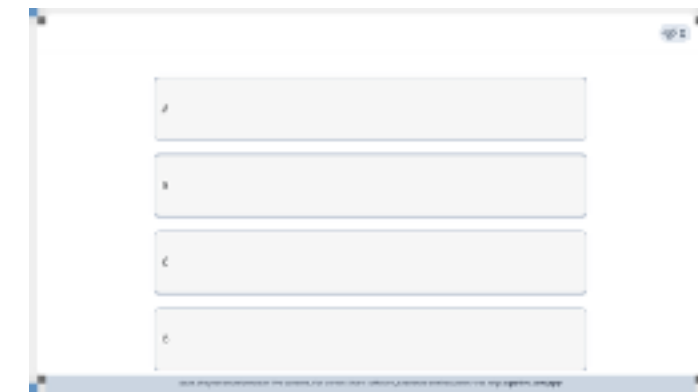
```
int main(int argc, char *argv[])
{
    int i,j;
    double **a;
    double sum=0, average;
    int dim=32768;
    if(argc < 2)
    {
        fprintf(stderr, "Usage: %s dimension\n",argv[0]);
        exit(1);
    }
    dim = atoi(argv[1]);
    a = (double **)malloc(sizeof(double *)*dim);
    for(i = 0 ; i < dim; i++)
        a[i] = (double *)malloc(sizeof(double)*dim);
    for(i = 0 ; i < dim; i++)
        for(j = 0 ; j < dim; j++)
            a[i][j] = rand();
    for(i = 0 ; i < dim; i++)
        for(j = 0 ; j < dim; j++)
            sum+=a[i][j];
    average = sum/(dim*dim);
    fprintf(stderr,"average: %lf\n",average);
    for(i = 0 ; i < dim; i++)
        free(a[i]);
    free(a);
    return 0;
}
```



What will happen?

- If we execute the code on the right-hand side code on a machine with only 16 GB of physical memory installed and the dim is "48000" (requires $48000 \times 48000 \times 8$ bytes ~ 18 GB memory at least), What will happen?
 - A. The program will crash in one of the malloc function call
 - B. The program will crash due to a "segmentation fault" that caused by accessing NULL pointer
 - C. The program will be killed automatically by the OS as it uses more than installed physical main memory
 - D. The program will finish without any issue

```
int main(int argc, char *argv[])
{
    int i,j;
    double **a;
    double sum=0, average;
    int dim=32768;
    if(argc < 2)
    {
        fprintf(stderr, "Usage: %s dimension\n",argv[0]);
        exit(1);
    }
    dim = atoi(argv[1]);
    a = (double **)malloc(sizeof(double *)*dim);
    for(i = 0 ; i < dim; i++)
        a[i] = (double *)malloc(sizeof(double)*dim);
    for(i = 0 ; i < dim; i++)
        for(j = 0 ; j < dim; j++)
            a[i][j] = rand();
    for(i = 0 ; i < dim; i++)
        for(j = 0 ; j < dim; j++)
            sum+=a[i][j];
    average = sum/(dim*dim);
    fprintf(stderr,"average: %lf\n",average);
    for(i = 0 ; i < dim; i++)
        free(a[i]);
    free(a);
    return 0;
}
```



What will happen?

- If we execute the code on the right-hand side code on a machine with only 16 GB of physical memory installed and the dim is "48000" (requires $48000 \times 48000 \times 8$ bytes ~ 18 GB memory at least), What will happen?
 - A. The program will crash in one of the malloc function call
 - B. The program will crash due to a "segmentation fault" that caused by accessing NULL pointer
 - C. The program will be killed automatically by the OS as it uses more than installed physical main memory
 - D. The program will finish without any issue

```
int main(int argc, char *argv[])
{
    int i,j;
    double **a;
    double sum=0, average;
    int dim=32768;
    if(argc < 2)
    {
        fprintf(stderr, "Usage: %s dimension\n",argv[0]);
        exit(1);
    }
    dim = atoi(argv[1]);
    a = (double **)malloc(sizeof(double *)*dim);
    for(i = 0 ; i < dim; i++)
        a[i] = (double *)malloc(sizeof(double)*dim);
    for(i = 0 ; i < dim; i++)
        for(j = 0 ; j < dim; j++)
            a[i][j] = rand();
    for(i = 0 ; i < dim; i++)
        for(j = 0 ; j < dim; j++)
            sum+=a[i][j];
    average = sum/(dim*dim);
    fprintf(stderr,"average: %lf\n",average);
    for(i = 0 ; i < dim; i++)
        free(a[i]);
    free(a);
    return 0;
}
```

Let's dig into this code

```
#define _GNU_SOURCE
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <sched.h>
#include <sys/syscall.h>
#include <time.h>

double a;

int main(int argc, char *argv[])
{
    int i, number_of_total_processes=4;
    number_of_total_processes = atoi(argv[1]);
    // Create processes
    for(i = 0; i< number_of_total_processes-1 && fork(); i++);
    // Assign the Process ID to a
    a = (int)getpid();
    fprintf(stderr, "\nProcess %d. Value of a is %lf and address of a is %p\n",getpid(), a, &a);
    sleep(10);
    fprintf(stderr, "\nProcess %d. Value of a is %lf and address of a is %p\n",getpid(), a, &a);
    return 0;
}
```



Consider the following code ...

- Consider the case when we run 4 instances of the given program at the same time on modern machines, which pair of statements is correct?

- ① The printed "address of a" is the same for every running instances
- ② The printed "address of a" is different for each instance
- ③ All running instances after the sleep will print the same value of a
- ④ Each instance after the sleep will print a its PID as a
- ⑤ Some instance will print the PID of the slowest process as a after the sleep

- A. (1) & (3)
- B. (1) & (4)
- C. (1) & (5)
- D. (2) & (3)
- E. (2) & (4)

```
#define _GNU_SOURCE
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <sched.h>
#include <sys/syscall.h>
#include <time.h>

double a;

int main(int argc, char *argv[])
{
    int i, number_of_total_processes=4;
    number_of_total_processes = atoi(argv[1]);
    for(i = 0; i< number_of_total_processes-1 && fork(); i++);
    a = (int)getpid();
    fprintf(stderr, "\nProcess %d. Value of a is %lf and address
of a is %p\n", getpid(), a, &a);
    sleep(10);
    fprintf(stderr, "\nProcess %d. Value of a is %lf and address
of a is %p\n", getpid(), a, &a);
    return 0;
}
```



Consider the following code ...

- Consider the case when we run 4 instances of the given program at the same time on modern machines, which pair of statements is correct?
 - ① The printed "address of a" is the same for every running instances
 - ② The printed "address of a" is different for each instance
 - ③ All running instances after the sleep will print the same value of a
 - ④ Each instance after the sleep will print a its PID as a
 - ⑤ Some instance will print the PID of the slowest process as a after the sleep

A. (1) & (3)
B. (1) & (4)
C. (1) & (5)
D. (2) & (3)
E. (2) & (4)

```
#define _GNU_SOURCE
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <sched.h>
#include <sys/syscall.h>
#include <time.h>

double a;

int main(int argc, char *argv[])
{
    int i, number_of_total_processes=4;
    number_of_total_processes = atoi(argv[1]);
    for(i = 0; i< number_of_total_processes-1 && fork(); i++);
    a = (int)getpid();
    fprintf(stderr, "\nProcess %d. Value of a is %lf and address  
of a is %p\n", getpid(), a, &a);
    sleep(10);
    fprintf(stderr, "\nProcess %d. Value of a is %lf and address  
of a is %p\n", getpid(), a, &a);
    return 0;
}
```



What will happen?

- If we execute the code on the right-hand side code on a machine with only 16 GB of physical memory installed and the dim is "48000" (requires $48000 \times 48000 \times 8$ bytes ~ 18 GB memory at least), What will happen?

A. The program will crash in one of the malloc function call

B. The program will crash due to the "segmentation fault" that caused by accessing NULL pointer

C. The program will be killed automatically by the OS as it uses more than installed physical main memory

D. The program will finish without any issue

```
int main(int argc, char *argv[])
{
    int i,j;
    double **a;
    double sum=0, average;
    int dim=32768;
    if(argc < 2)
    {
        fprintf(stderr, "Usage: %s dimension\n",argv[0]);
        exit(1);
    }
    dim = atoi(argv[1]);
    a = (double **)malloc(sizeof(double *)*dim);
    for(i = 0 ; i < dim; i++)
        a[i] = (double *)malloc(sizeof(double)*dim);
    for(i = 0 ; i < dim; i++)
        for(j = 0 ; j < dim; j++)
            a[i][j] = rand();
    for(i = 0 ; i < dim; i++)
        for(j = 0 ; j < dim; j++)
            sum+=a[i][j];
    average = sum/(dim*dim);
    fprintf(stderr,"average: %lf\n",average);
    for(i = 0 ; i < dim; i++)
        free(a[i]);
    free(a);
    return 0;
}
```


Consider the following code ...

- Consider the case when we run 4 instances of the given program at the same time on modern machines, which pair of statements is correct?

- ① The printed "address of a" is the same for every running instances
- ② The printed "address of a" is different for each instance
- ③ All running instances after the sleep will print the same value of a
- ④ Each instance after the sleep will print a its PID as a
- ⑤ Some instance will print the PID of the slowest process as a after the sleep

- A. (1) & (3)
- B. (1) & (4)
- C. (1) & (5)
- D. (2) & (3)
- E. (2) & (4)

```
#define _GNU_SOURCE
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <sched.h>
#include <sys/syscall.h>
#include <time.h>

double a;

int main(int argc, char *argv[])
{
    int i, number_of_total_processes=4;
    number_of_total_processes = atoi(argv[1]);
    for(i = 0; i< number_of_total_processes-1 && fork(); i++);
    a = (int)getpid();
    fprintf(stderr, "\nProcess %d. Value of a is %lf and address
of a is %p\n", getpid(), a, &a);
    sleep(10);
    fprintf(stderr, "\nProcess %d. Value of a is %lf and address
of a is %p\n", getpid(), a, &a);
    return 0;
}
```

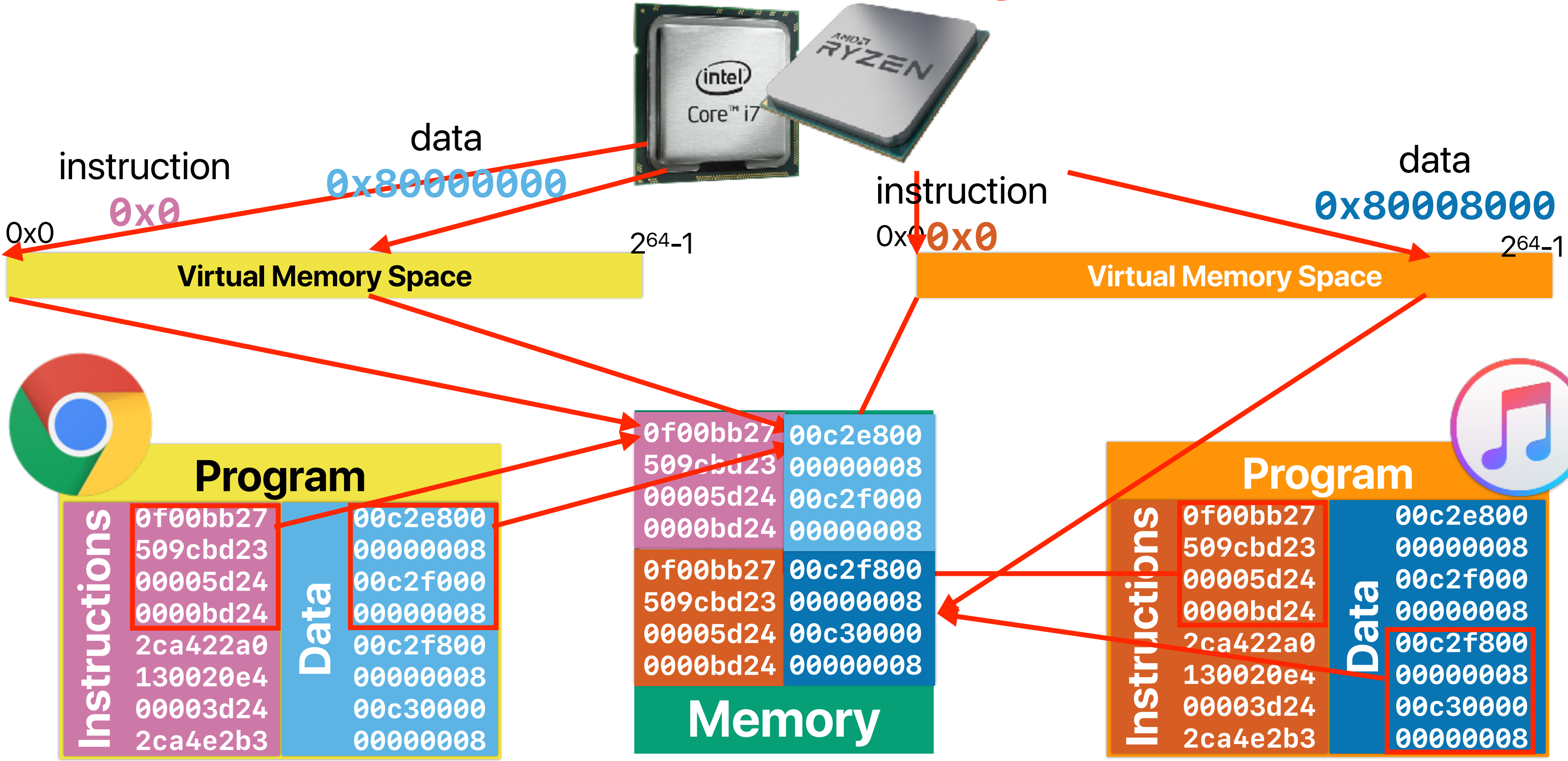
**The system allows a
multiple programs to use
the same memory address!**

Outline

- Virtual memory
- Architectural support for virtual memory

Virtual Memory

Virtual memory



Demo revisited

&a = 0x5da1e73ef030

Process A

**Process A's
Virtual
Memory Space**

Process B

**Process B's
Virtual
Memory Space**

```
#define _GNU_SOURCE
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <sched.h>
#include <sys/syscall.h>
#include <time.h>
#include <stdint.h>
```

```
double a = 0;
```

```
int main(int argc, char *argv[])
{
    uint64_t i, number_of_total_processes=4, p;
    if(argc < 2)
    {
        fprintf(stderr, "Usage: %s number_of_processes\n", argv[0]);
        exit(1);
    }
    number_of_total_processes = atoi(argv[1]);
    for(i = 0; i < number_of_total_processes-1 && fork(); i++);
    srand((int)time(NULL)+(int)getpid());
    a = rand();
    fprintf(stderr, "\nProcess %d: Value of a is %lf and address of a is %p\n", (int)getpid(), a, &a);
    sleep(10);
    fprintf(stderr, "\n10 Seconds Later -- Process %d: Value of a is %lf and address of a is %p\n", (int)getpid(), a, &a);
    return 0;
}
```

Why Virtual memory?

- Allowing multiple applications to share physical main memory
 - Memory protection/isolation among programs/processes is automatically achieved
- Allowing applications to work even the installed physical memory or available physical memory is smaller than the working set of the application
 - Programmer does not need to worry about the physical memory capacity of different machines — make compiled program compatible
 - Multiple programs can work concurrently even though their total memory demand is larger than the installed physical memory



Program A

0f00bb27	00c2e800
509cbd23	00000008
00005d24	00c2f000
0000bd24	00000008
2ca422a0	00c2f800
130020e4	00000008
00003d24	00c30000
2ca4e2b3	00000008

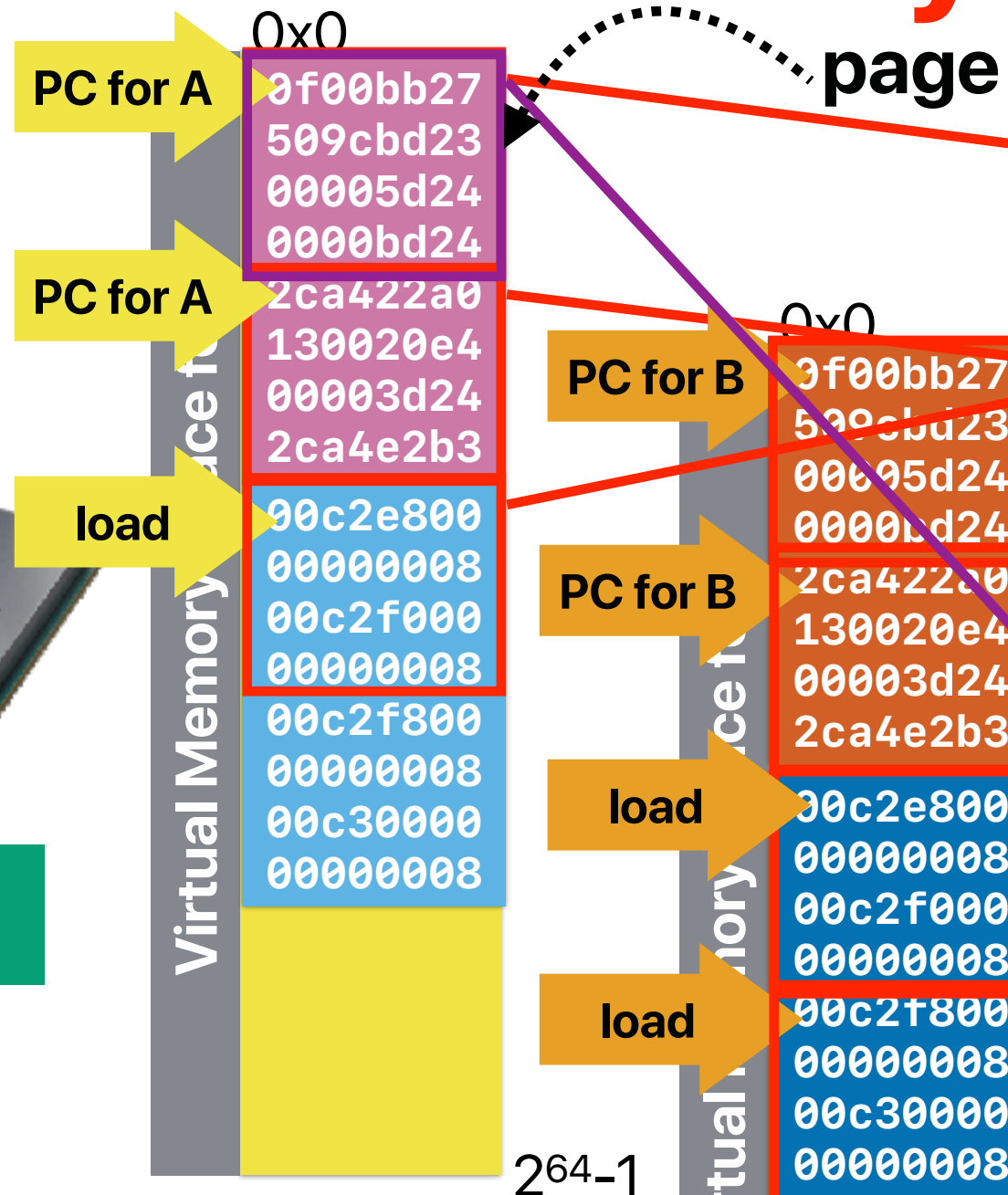


Program B

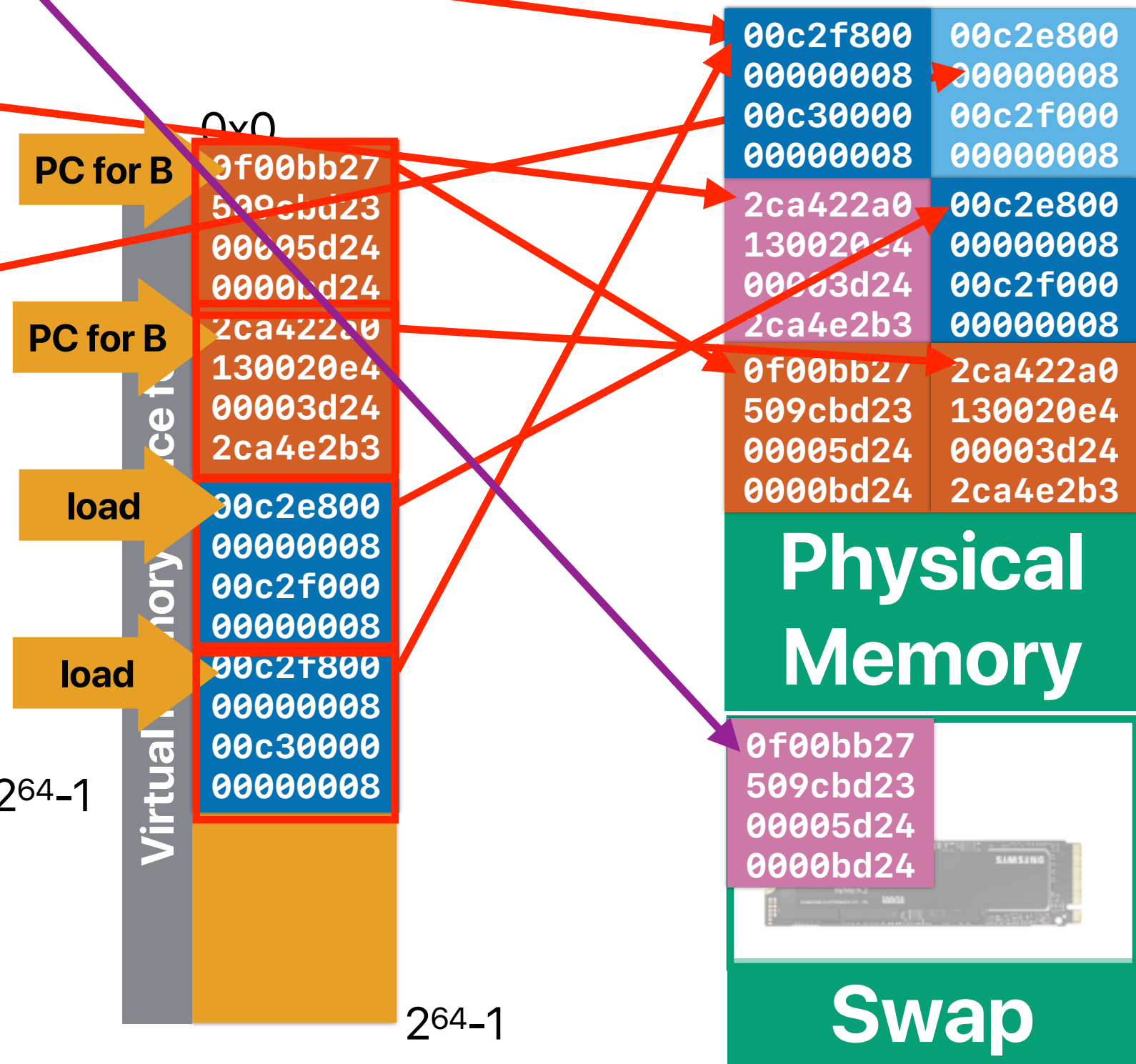
0f00bb27	00c2e800
509cbd23	00000008
00005d24	00c2f000
0000bd24	00000008
2ca422a0	00c2f800
130020e4	00000008
00003d24	00c30000
2ca4e2b3	00000008



Virtual memory



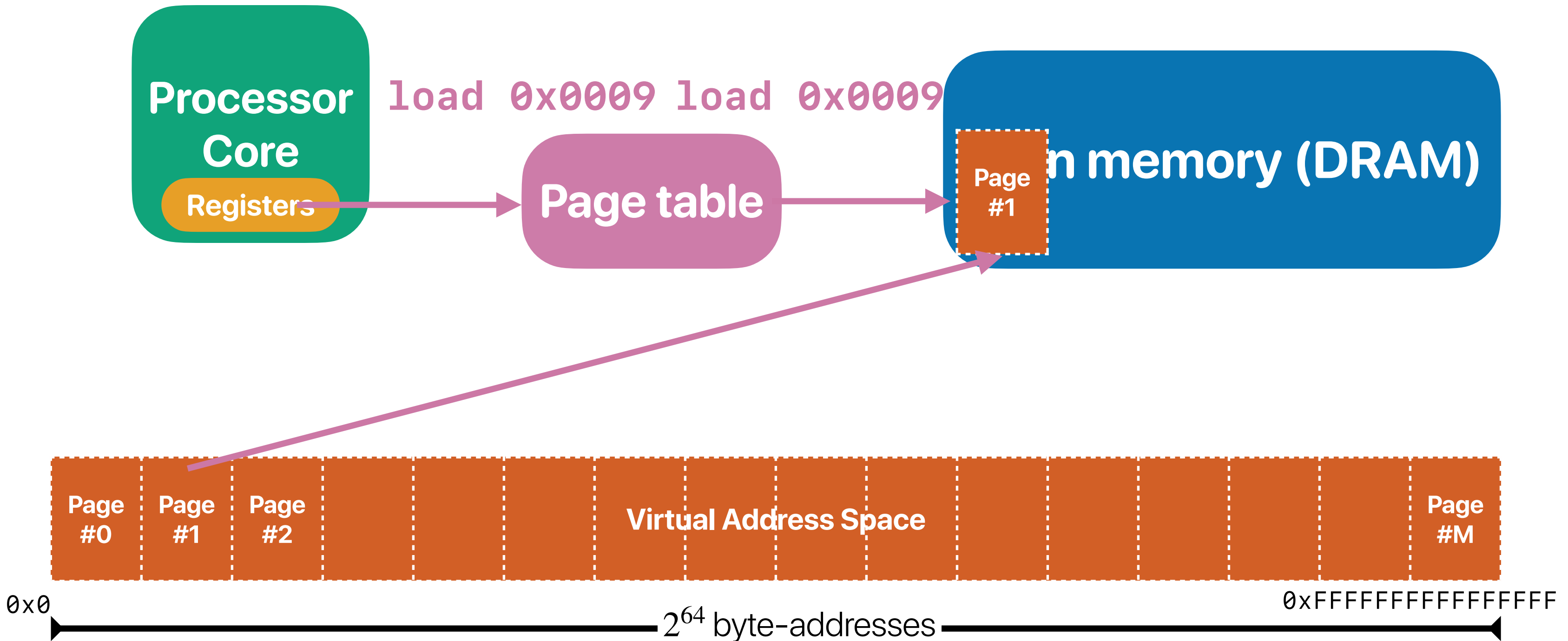
This approach is called demand paging + swapping



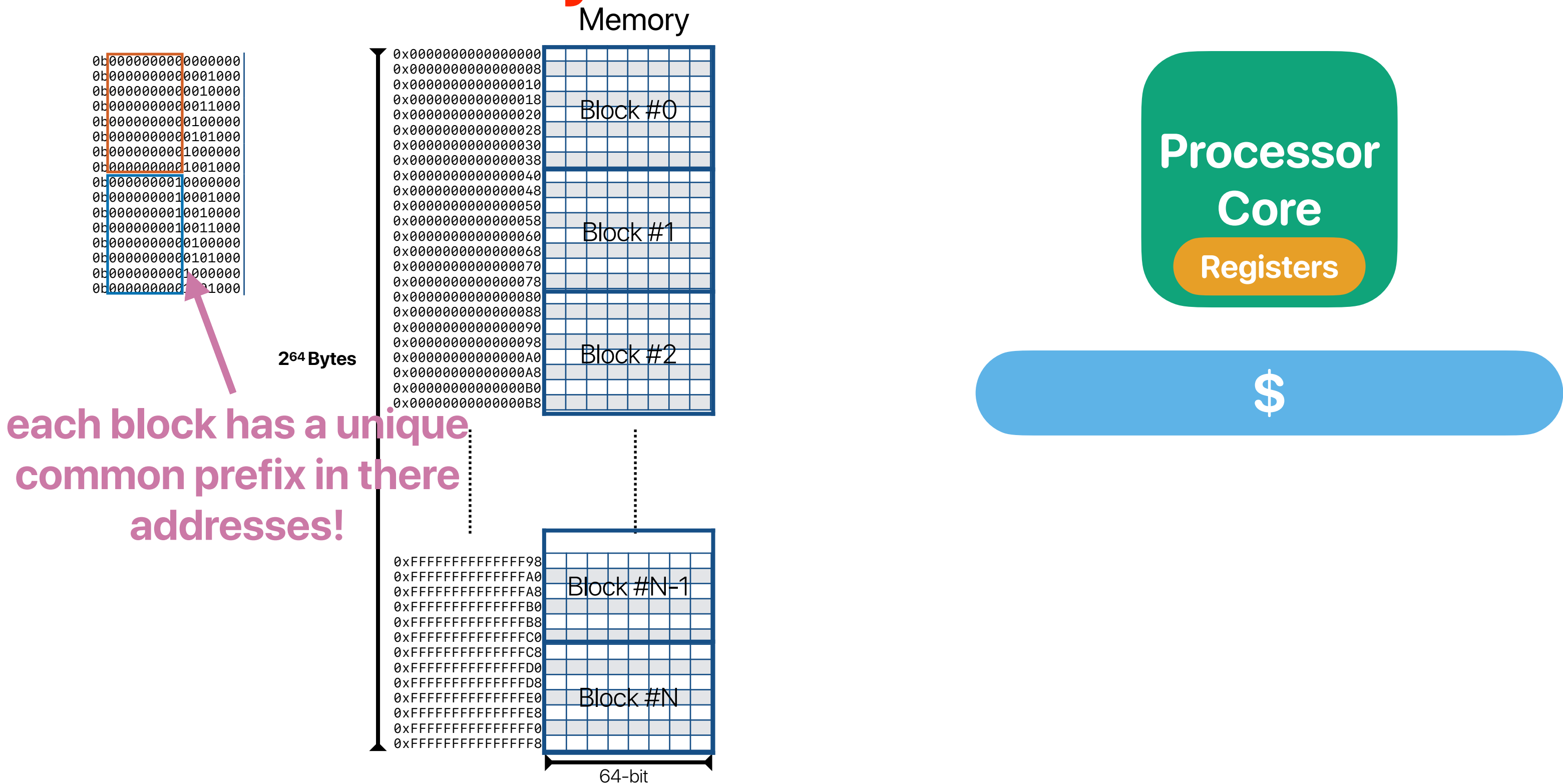
Virtual memory

- An **abstraction** of memory space available for programs/software/programmer
- Programs execute using virtual memory address
- The operating system and hardware work together to handle the mapping between virtual memory addresses and real/physical memory addresses
- Virtual memory organizes memory locations into "**pages**"

Demand paging — another angle



Partition memory addresses into fix-sized chunks



Demand paging + Swapping

- **Paging:** partition virtual/physical memory spaces into fix-sized pages
- **Page fault:** when the requested page cannot be found in the physical memory — created the demand of allocating pages!
- **Demand paging:** Allocate a physical memory page for a virtual memory page when the virtual page is needed (page fault occurs)
- **Swapping:** use secondary storage to store pages not in DRAM

Demand paging + Swapping v.s. caching

- Treating physical main memory as a “cache” of virtual memory
- The block size is the “page size”
- The page table is the “tag array”
- It’s a “fully-associate” cache — a virtual page can go anywhere in the physical main memory
- The storage serves as the lower level memory hierarchy for physical main memory

Virtual memory

- An **abstraction** of memory space available for programs/software/programmer
- Programs execute using virtual memory address
- The operating system and hardware work together to handle the mapping between virtual memory addresses and real/physical memory addresses
- Virtual memory organizes memory locations into "**pages**"



Processor Core

Registers

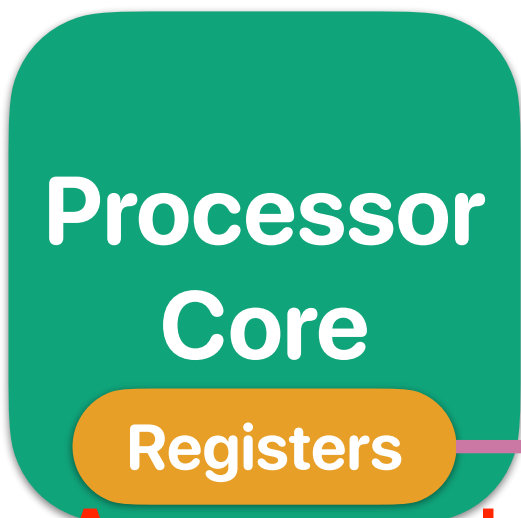
Registers

load 0x0009

Page table

Main memory (DRAM)

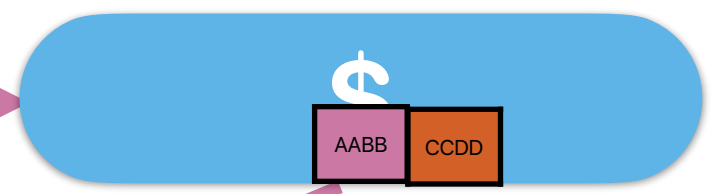




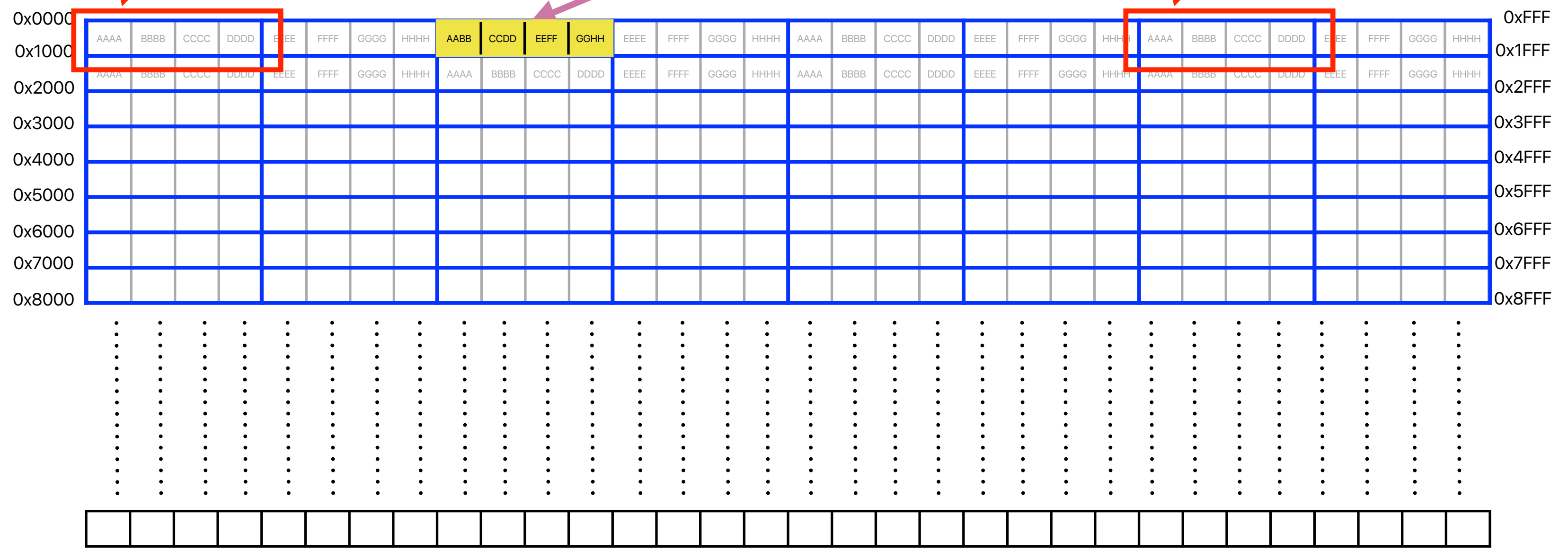
Recap: To capture "spatial" locality, \$ fetch a "block"

lw 0x0024

Assume each block is 16 bytes



"Logically" partition memory space into "blocks"



Why Virtual memory?

- Allowing multiple applications to share physical main memory
 - Memory protection/isolation among programs/processes is automatically achieved
- Allowing applications to work even the installed physical memory or available physical memory is smaller than the working set of the application
 - Programmer does not need to worry about the physical memory capacity of different machines — make compiled program compatible
 - Multiple programs can work concurrently even though their total memory demand is larger than the installed physical memory

Mechanism: Demand paging

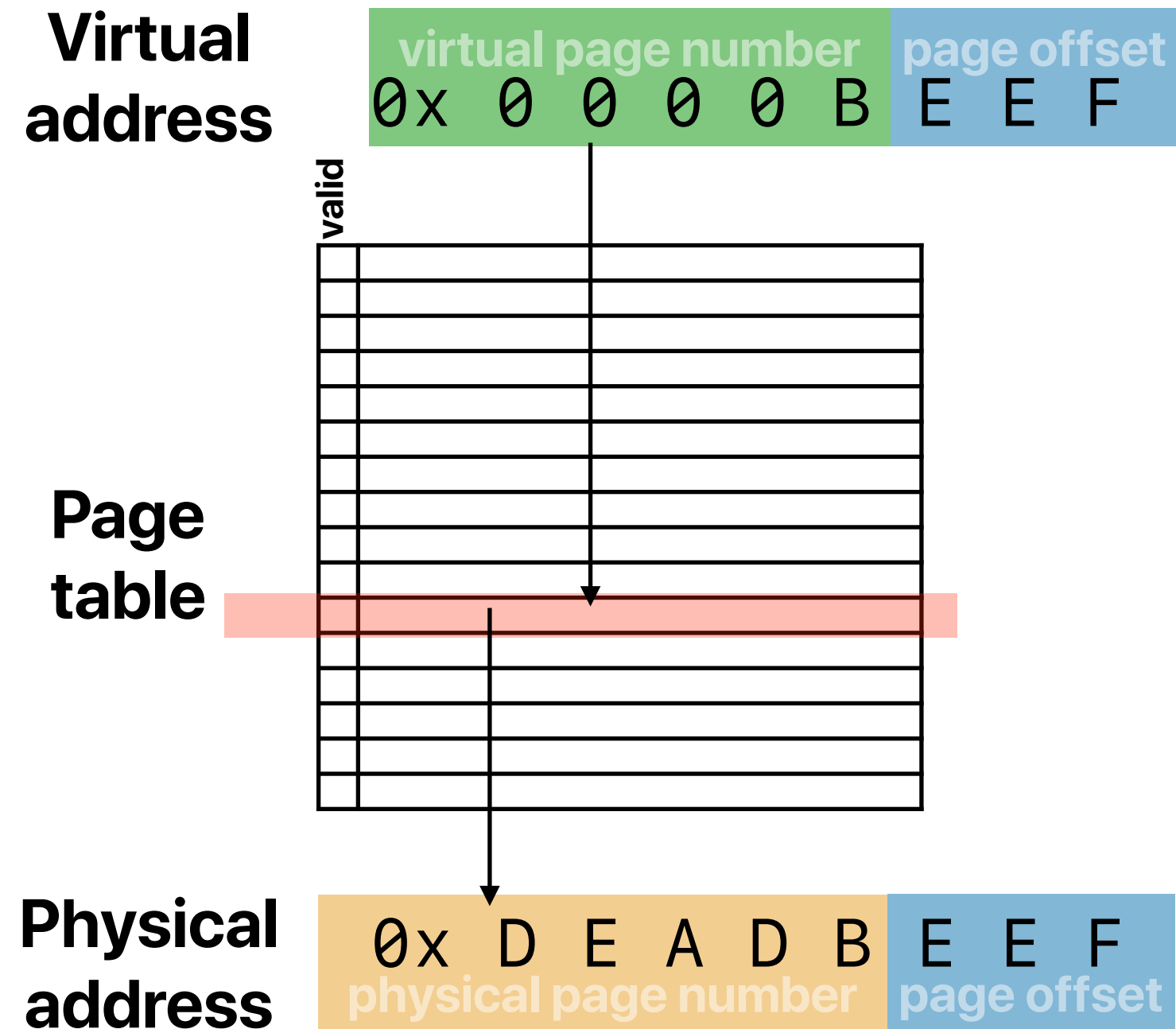
- Treating physical main memory as a “cache” of virtual memory
- The block size is the “page size”
- The page table is the “tag array”
- It’s a “fully-associate” cache — a virtual page can go anywhere in the physical main memory
- The storage serves as the lower level memory hierarchy for physical main memory

Takeaways: Virtual Memory

- Virtual memory is essential to support the success of software industry

Address translation

- Processor receives virtual addresses from the running code, main memory uses physical memory addresses
- Virtual address space is organized into "pages"
- The system references the **page table** to translate addresses
 - Each process has its own page table
 - The page table content is maintained by OS





Size of page table

- Assume that we have **64-bit** virtual address space, each page is 4KB, each page table entry is 8 Bytes, what magnitude in size is the page table for a process?
 - A. MB — 2^{20} Bytes
 - B. GB — 2^{30} Bytes
 - C. TB — 2^{40} Bytes
 - D. PB — 2^{50} Bytes
 - E. EB — 2^{60} Bytes

A	
B	
C	
D	
E	

Size of page table

- Assume that we have **64-bit** virtual address space, each page is 4KB, each page table entry is 8 Bytes, what magnitude in size is the page table for a process?

A. MB — 2^{20} Bytes

B. GB — 2^{30} Bytes

C. TB — 2^{40} Bytes

D. PB — 2^{50} Bytes

E. EB — 2^{60} Bytes

$$\frac{2^{64} \text{ Bytes}}{4 \text{ KB}} \times 8 \text{ Bytes} = 2^{55} \text{ Bytes} = 32 \text{ PB}$$

If you still don't know why — you need to take CS202

Conventional page table

0x0

0xFFFFFFFFFFFFFFFF

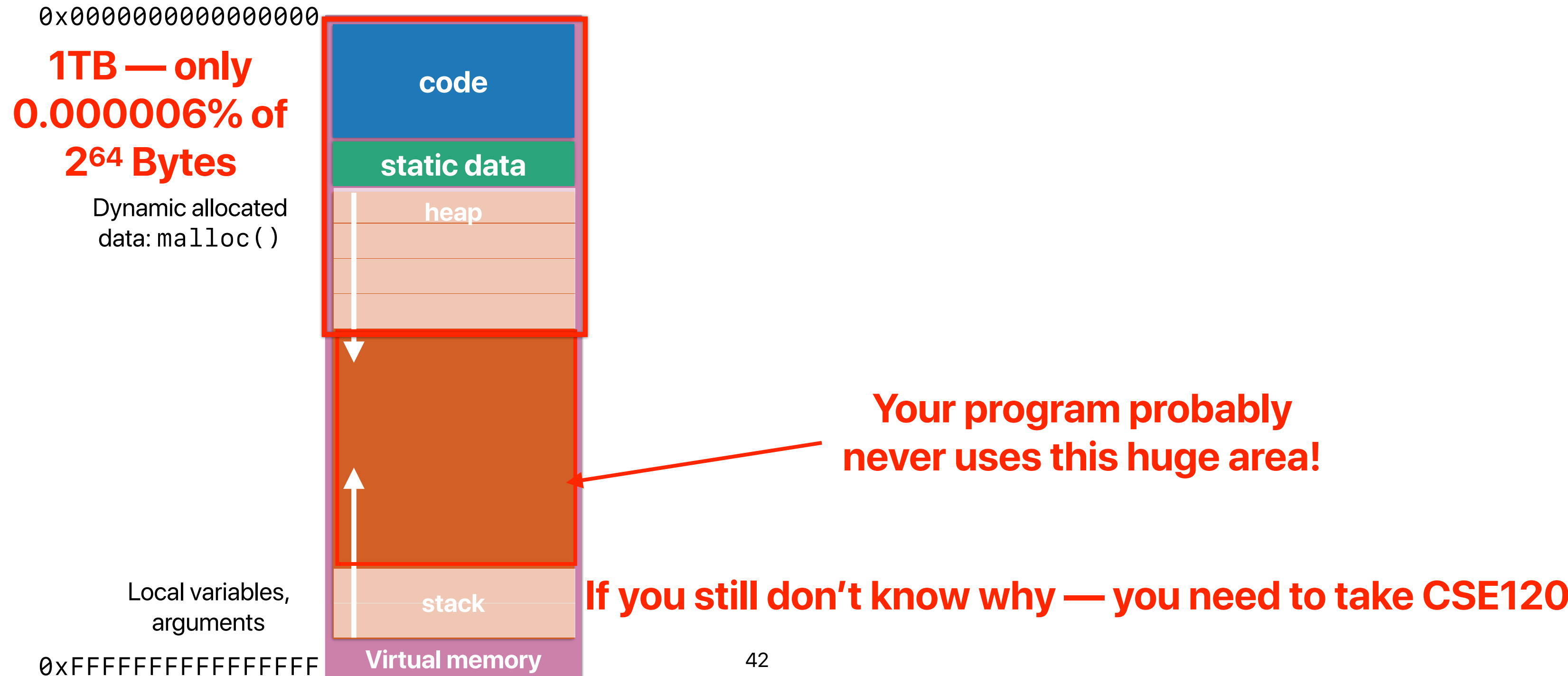
Virtual Address Space

- must be consecutive in the physical memory
- need a big segment! — difficult to find a spot
- simply too big to fit in memory if address space is large!

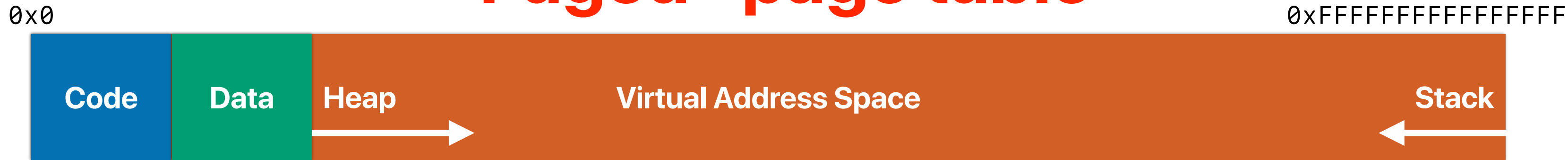
$\frac{2^{64} B}{2^{12} B}$ page table entries/leaf nodes



Do we really need a large table?



"Paged" page table

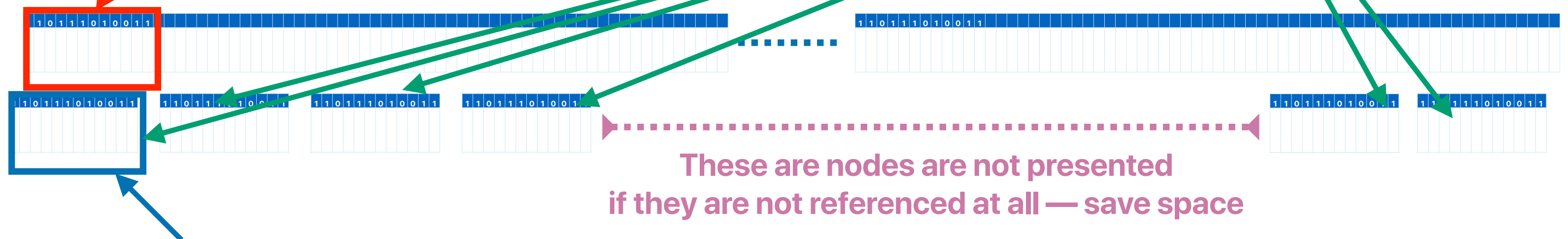


Break up entries into pages!
Each of these occupies exactly a page

$$\frac{2^{12} B}{2^3 B} = 2^9 \text{ PTEs per node}$$

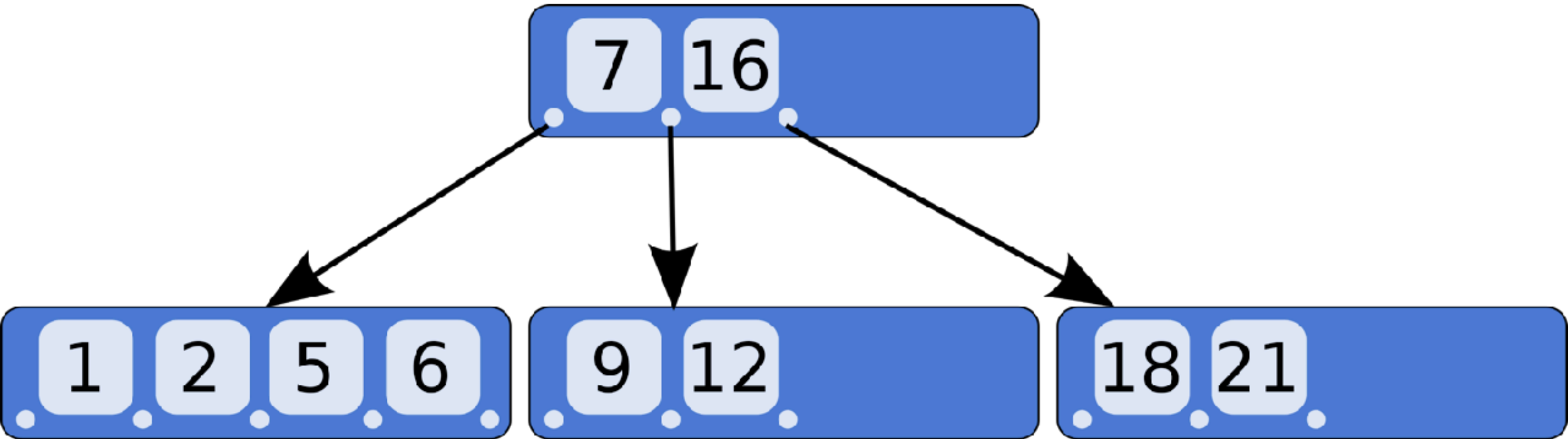
Otherwise, you always need to find more than one consecutive pages — difficult!

Question:
These nodes are spread out,
how to locate them in the memory?



Allocate page table entry nodes "on demand"

B-tree

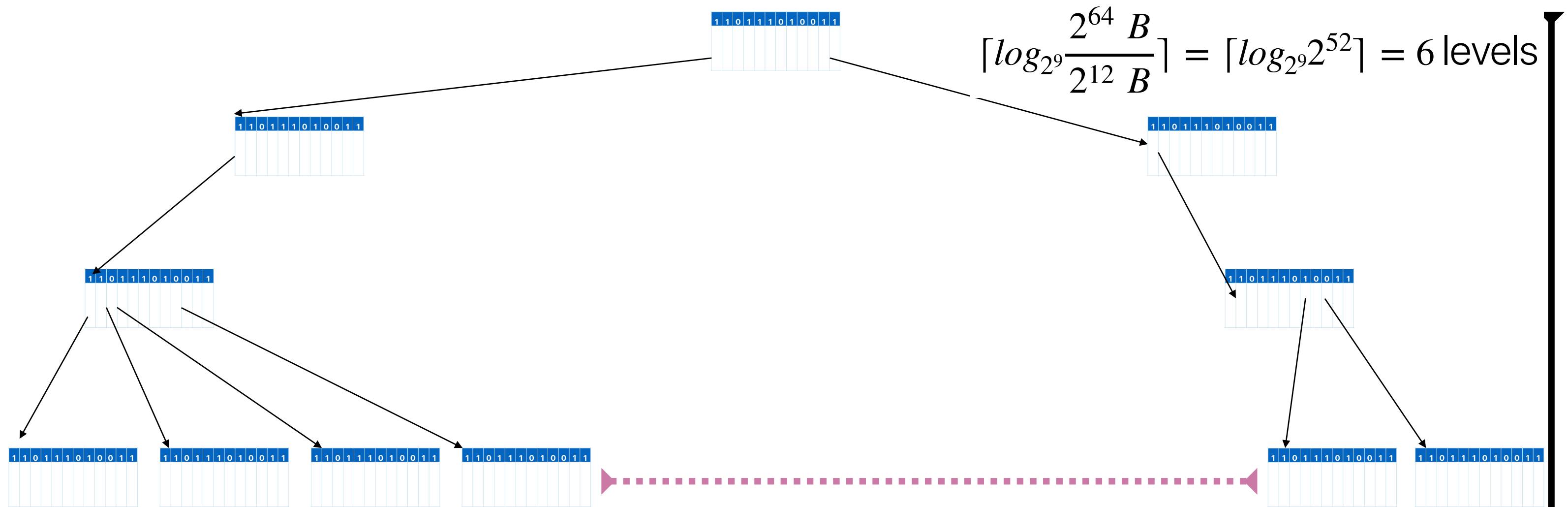
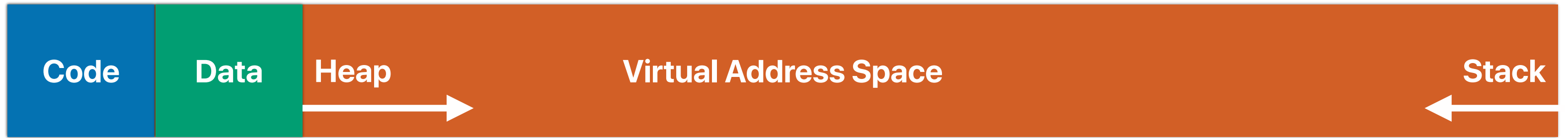


<https://en.wikipedia.org/wiki/B-tree#/media/File:B-tree.svg>

Hierarchical Page Table

0x0

0xFFFFFFFFFFFFFFFF

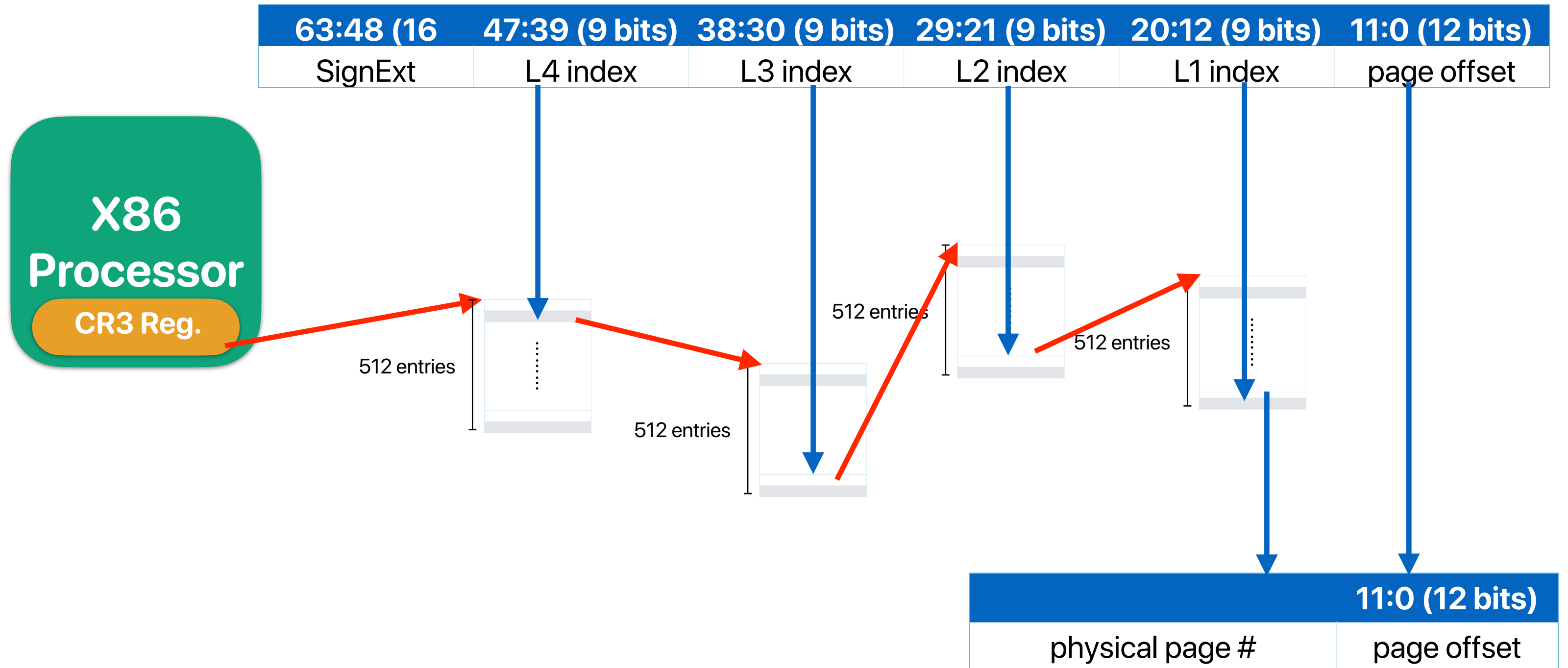


$$\lceil \log_2 \frac{2^{64} B}{2^{12} B} \rceil = \lceil \log_2 2^{52} \rceil = 6 \text{ levels}$$

These are nodes are not presented
as they are not referenced at all.

$\frac{2^{64} B}{2^{12} B}$ page table entries/leaf nodes (worst case)

Address translation in x86-64



Takeaways: Virtual Memory

- Virtual memory is essential to support the success of software industry
- To reduce the page table size, we introduced hierarchical page table data structure

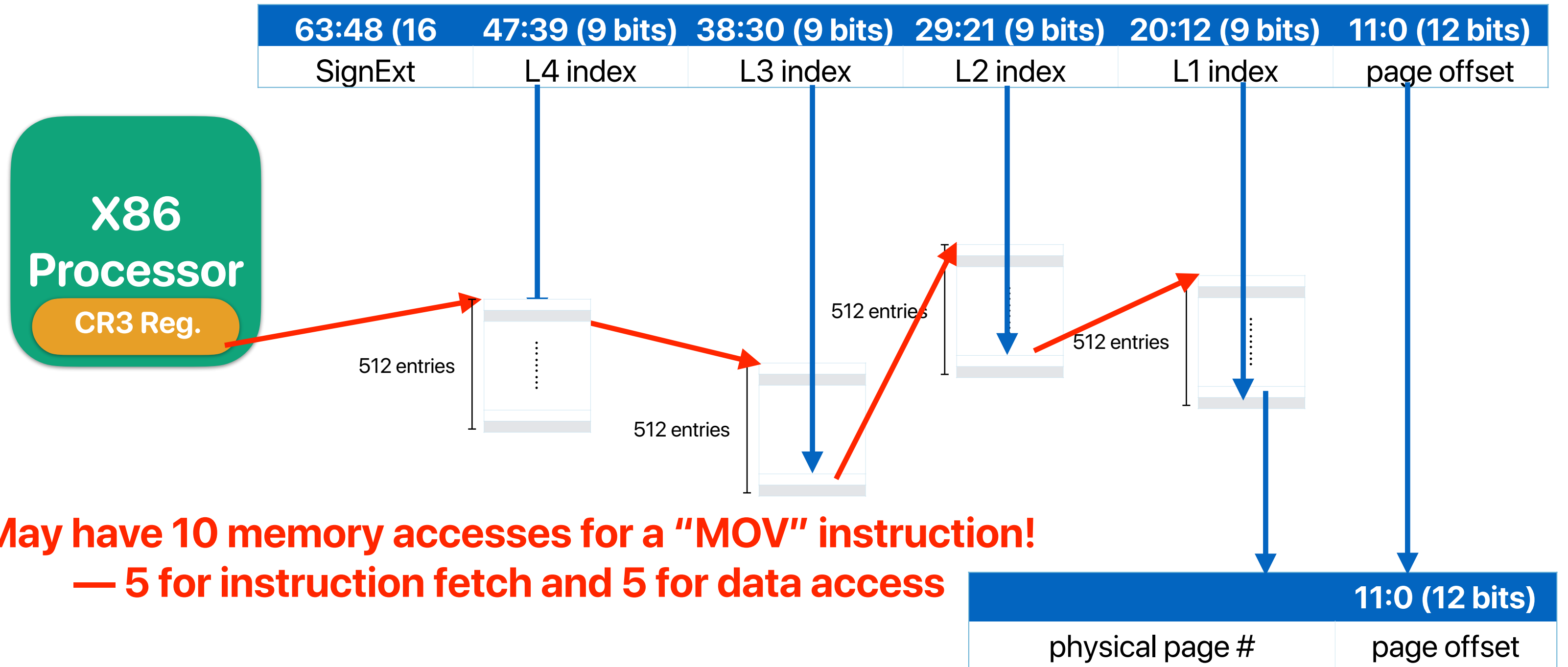


When we have virtual memory...

- If an x86 processor supports virtual memory through the basic format of the page table as shown in the previous slide, how many memory accesses can a **mov** instruction that access data memory once incur?
- A. 2
 - B. 4
 - C. 6
 - D. 8
 - E. 10

A
B
C
D
E

Address translation in x86-64

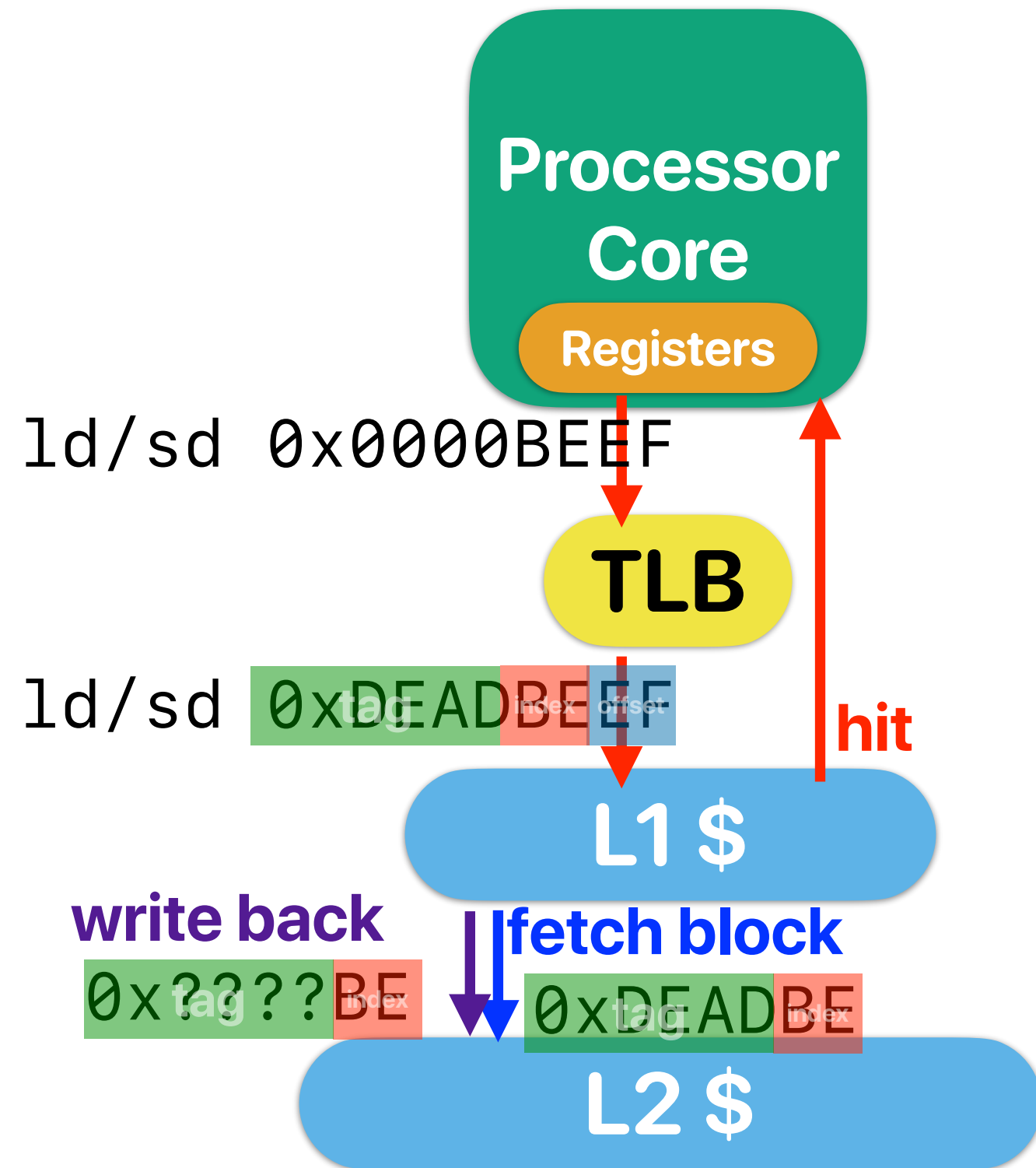


When we have virtual memory...

- If an x86 processor supports virtual memory through the basic format of the page table as shown in the previous slide, how many memory accesses can a **mov** instruction that access data memory once incur?
 - A. 2
 - B. 4
 - C. 6
 - D. 8
 - E. 10

Avoiding the address translation overhead

TLB: Translation Look-aside Buffer



- TLB — a small SRAM stores frequently used page table entries
- Good — A lot faster than having everything going to the DRAM
- Bad — Still on the critical path

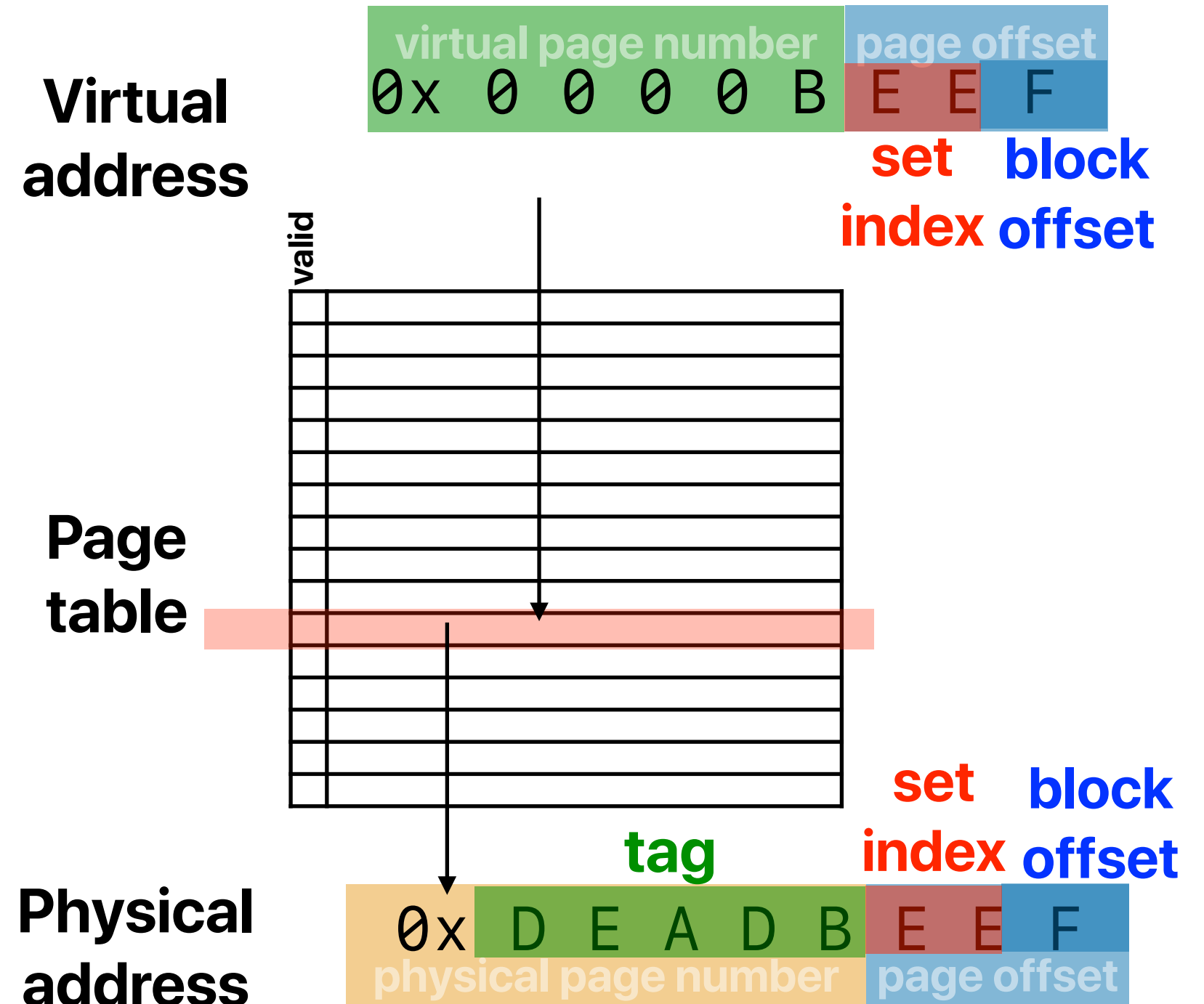
TLB + Virtual cache

- L1 \$ accepts virtual address — you don't need to translate
- Good — you can access both TLB and L1-\$ at the same time and physical address is only needed if L1-\$ misses
- Bad — it doesn't work in practice
 - Many applications have the same virtual address but should be pointing different **physical addresses**
 - An application can have "aliasing virtual addresses" pointing to the same **physical address**



Virtually indexed, physically tagged cache

- Can we find physical address directly in the virtual address — Not everything — but the page offset isn't changing!
- Can we indexing the cache using the "partial physical address"?
 - Yes — Just make set index + block set to be exactly the page offset



Virtually indexed, physically tagged cache

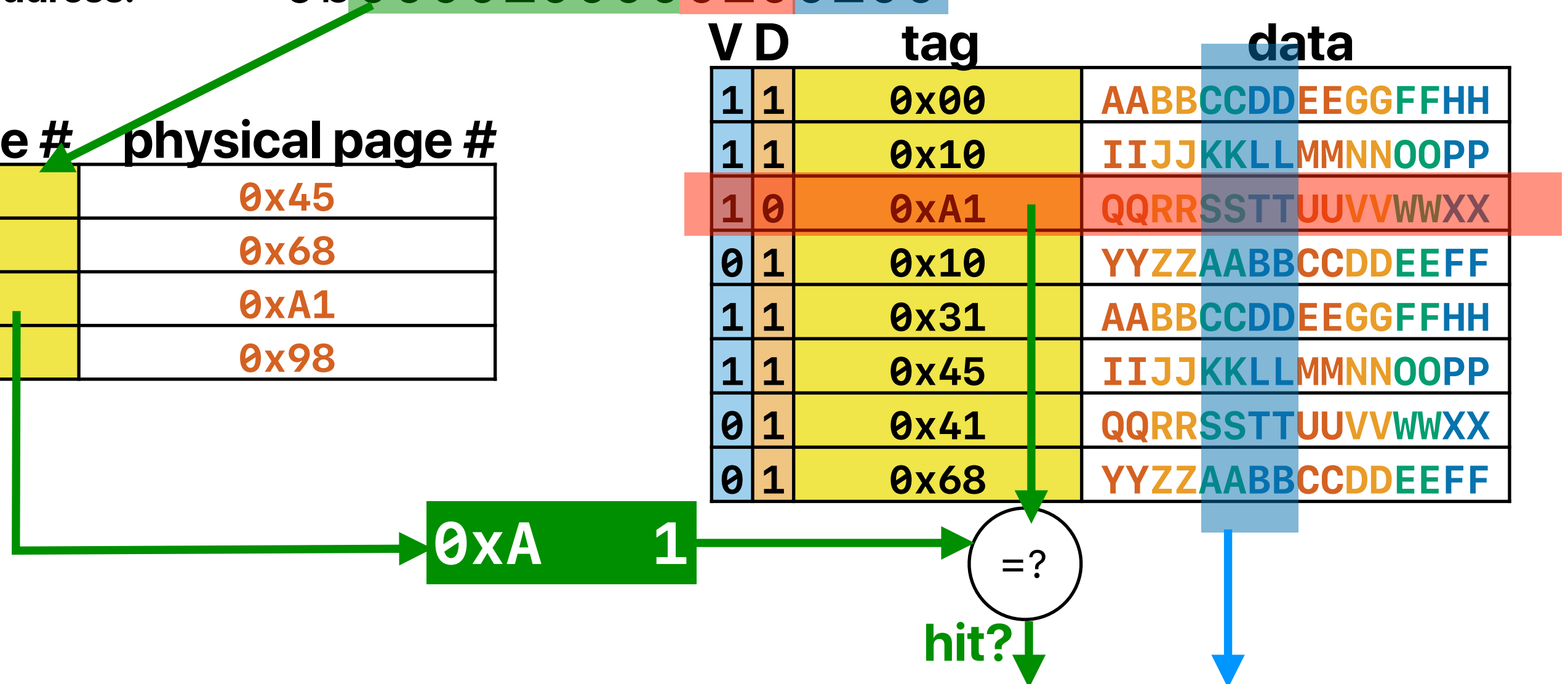
memory address:

0x0		8		2		4
				set		block

memory address: 0b0000100000100100

V	virtual page #	physical page #
1	0x29	0x45
1	0xDE	0x68
1	0x10	0xA1
0	0x8A	0x98

V D		tag	data
1	1	0x00	AABBCCDDEEGGFFHH
1	1	0x10	IIJJKKLLMMNNOOPP
1	0	0xA1	QQRRSSTTUUVVWWXX
0	1	0x10	YYZZAABBCCDDEEFF
1	1	0x31	AABBCCDDEEGGFFHH
1	1	0x45	IIJJKKLLMMNNOOPP
0	1	0x41	QQRRSSTTUUVVWWXX
0	1	0x68	YYZZAABBCCDDEEFF



Virtually indexed, physically tagged cache

- If page size is 4KB —

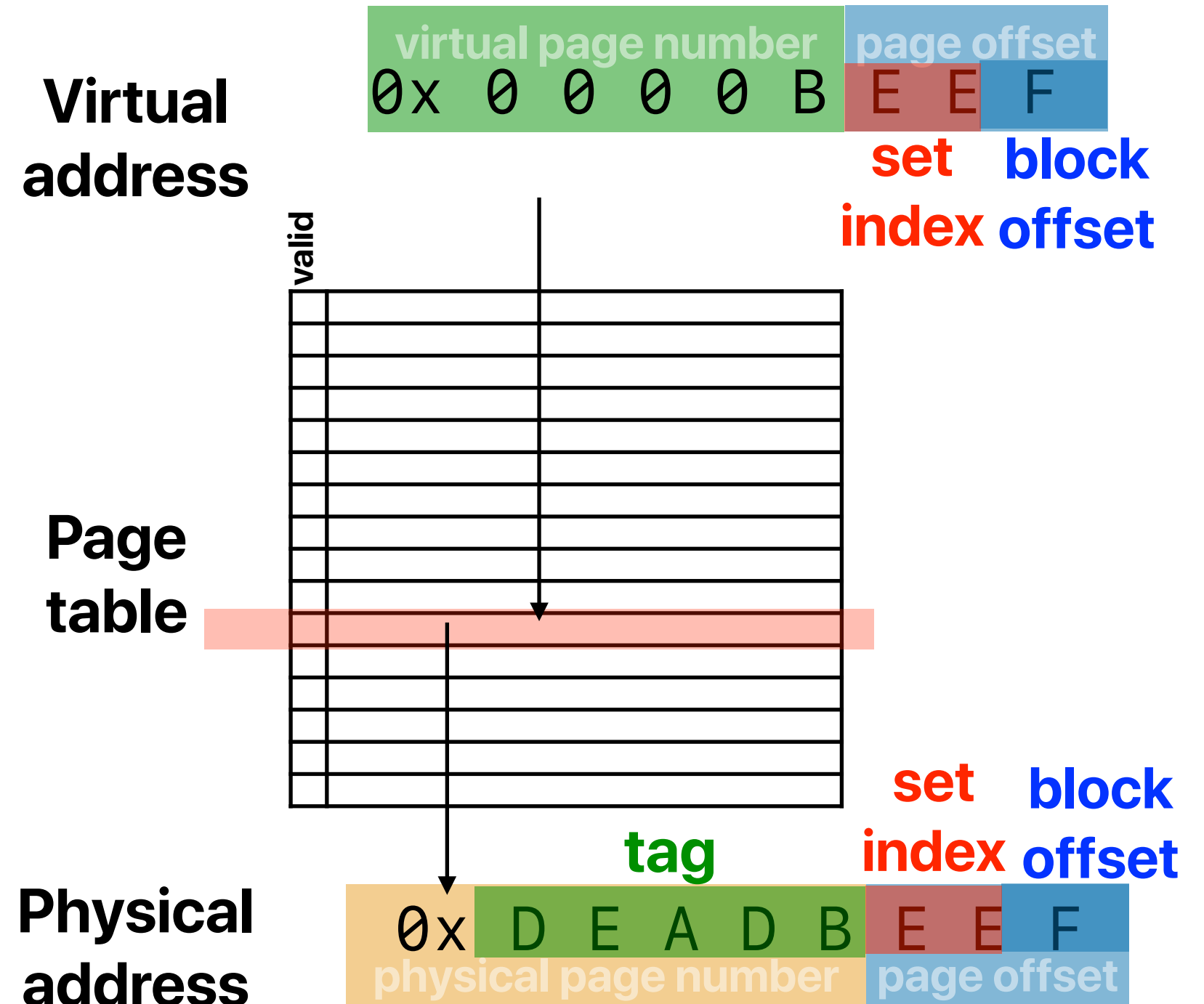
$$lg(B) + lg(S) = lg(4096) = 12$$

$$C = ABS$$

$$C = A \times 2^{12}$$

if $A = 1$

$$C = 4KB$$



Virtually indexed, physically tagged cache

The Core i7 13700

```
Index*  
che/index0/physical_line_partition:1  
che/index0/number_of_sets:64  
che/index0/ways_of_associativity:12  
che/index0/id:0  
che/index0/shared_cpu_list:0-1  
che/index0/type:Data  
che/index0/size:48K  
che/index0/level:1  
che/index0/coherency_line_size:64  
che/index0/shared_cpu_map:000003  
che/index1/physical_line_partition:1
```

- If page size is 4KB —

$$\lg(B) + \lg(S) = \lg(4096) = 12$$

$$C = ABS$$

$$C = A \times 2^{12}$$

$$\text{if } A = 12$$

$$C = 48 \text{ KB}$$



Virtual indexed, physical tagged cache limits the cache size

- If you want to build a virtual indexed, physical tagged cache with 32KB capacity, which of the following configuration is possible? Assume the operating system use 4K pages.
 - A. 32B blocks, 2-way
 - B. 32B blocks, 4-way
 - C. 64B blocks, 4-way
 - D. 64B blocks, 8-way

A poll interface with four options labeled A, B, C, and D, each in a separate input box. The options correspond to the multiple-choice question on the slide.

Virtual indexed, physical tagged cache limits the cache size

- If you want to build a virtual indexed, physical tagged cache with 32KB capacity, which of the following configuration is possible? Assume the operating system use 4K pages.

A. 32B blocks, 2-way

B. 32B blocks, 4-way

C. 64B blocks, 4-way

D. 64B blocks, 8-way

Exactly how the Sky Lake processor used in our assignment its own cache

$$\lg(B) + \lg(S) = \lg(4096) = 12$$

$$C = ABS$$

$$32KB = A \times 2^{12}$$

$$A = 8$$

Takeaways: Virtual Memory

- Virtual memory is essential to support the success of software industry
- To reduce the page table size, we introduced hierarchical page table data structure
- Virtually-indexed, physically tagged cache provides the efficiency for accessing cache and TLB together — but limited cache design

Announcement

- Reading quiz #5 due tomorrow — take on Gradescope
- Assignment #3 released — due this weekend
- Please find assignment #1 and #2 solution on Piazza
- Don't forget to schedule & take your midterm before 8/21
 - Check the slide on 8/15/2025 for more details

Computer Science & Engineering

142

つづく

